

Guía Detallada de Tests Unitarios con JUnit 5

¿Qué son los tests unitarios y por qué son fundamentales?

Los tests unitarios son programas automatizados diseñados para verificar el comportamiento correcto de unidades mínimas de código, típicamente métodos o funciones individuales. A diferencia de la verificación manual —donde se ejecuta el programa y se inspecciona visualmente el resultado—, los tests unitarios validan sistemáticamente que cada componente funcione según lo especificado, incluso ante cambios futuros en la implementación. Esta práctica constituye la base de la ingeniería de software moderna, ya que permite detectar regresiones de forma inmediata y facilita la refactorización con confianza.

Desde una perspectiva de encapsulación y diseño robusto, los tests unitarios actúan como especificaciones ejecutables del comportamiento público de una clase. Al definir qué resultados debe producir un método ante determinadas entradas, se establece un contrato implícito que la implementación debe cumplir. Esto refuerza la separación entre interfaz e implementación: mientras la interfaz pública permanezca estable, la lógica interna puede evolucionar libremente siempre que los tests continúen pasando, garantizando así la preservación de invariantes y la coherencia del sistema.

Configuración de JUnit 5 en proyectos Maven

Para incorporar JUnit 5 en un proyecto gestionado con Maven, es necesario declarar la dependencia correspondiente en el archivo `pom.xml`. Esta dependencia pertenece al ámbito `test`, lo que significa que solo estará disponible durante la fase de pruebas y no se incluirá en el artefacto final de producción. La configuración mínima requerida es:

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Es fundamental respetar la estructura del POM: esta sección debe situarse después de `<properties>` y antes del cierre de `</project>`. Una vez añadida la dependencia, Maven descargará automáticamente las bibliotecas necesarias al ejecutar `mvn test` o al actualizar el proyecto en el IDE. Esta separación entre código de producción (`src/main/java`) y código de pruebas (`src/test/java`) es un principio arquitectónico clave que refuerza la encapsulación y evita dependencias circulares.

Estructura y anatomía de una clase de test

Las clases de test en JUnit 5 siguen una estructura estandarizada que facilita su identificación y ejecución automática. Deben residir en el paquete `src/test/java`, preferiblemente replicando la estructura de paquetes del código bajo prueba. Por ejemplo, si la clase `Calculadora` está en `ejercicio4`, su test correspondiente `CalculadoraTest` también debe declarar `package ejercicio4;`.

```
package ejercicio4;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {

    @Test
    void testSumar() {
        // Arrange: Preparación del entorno
        Calculadora calc = new Calculadora();

        // Act: Ejecución de la operación bajo prueba
        int resultado = calc.sumar(2, 3);

        // Assert: Verificación del resultado esperado
        assertEquals(5, resultado, "2 + 3 debe ser 5");
    }
}
```

El ciclo AAA (Arrange-Act-Assert) es un patrón esencial para la legibilidad y mantenibilidad de los tests. En la fase *Arrange* se configuran los objetos y datos necesarios; en *Act* se invoca el método bajo prueba; y en *Assert* se validan los resultados mediante aserciones. El uso de nombres descriptivos para los métodos de test (ej. `testSumarDosPositivos`) mejora significativamente la diagnosticabilidad cuando un test falla, ya que el nombre mismo describe el escenario verificado.

Aserciones: el mecanismo de verificación

Las aserciones son las instrucciones que comparan el comportamiento real del código con el comportamiento esperado. JUnit 5 proporciona una biblioteca rica de métodos estáticos en `org.junit.jupiter.api.Assertions`, accesibles mediante `import static` para una sintaxis concisa. Las aserciones básicas incluyen:

- `assertEquals(expected, actual)`: Verifica igualdad de valores. Para tipos primitivos compara el contenido; para objetos, el comportamiento depende de si se ha sobrescrito `equals()`.
- `assertTrue(condition)` / `assertFalse(condition)`: Valida expresiones booleanas.
- `assertNull(object)` / `assertNotNull(object)`: Comprueba la presencia o ausencia de referencia nula.
- `assertThrows(expectedType, executable)`: Confirma que se lanza una excepción específica ante condiciones inválidas.

Para números en coma flotante, existe una variante especializada `assertEquals(expected, actual, delta)` donde `delta` define el margen de error aceptable debido a la imprecisión inherente de la representación binaria. Por ejemplo, `assertEquals(0.3, 0.1 + 0.2, 0.001)` acepta una diferencia máxima de 0.001. Este enfoque refleja una comprensión profunda de las limitaciones numéricas y es esencial para tests robustos en cálculos científicos o financieros.

Ejecución y diagnóstico de tests

Los tests pueden ejecutarse mediante la línea de comandos con `mvn test`, que compila tanto el código de producción como el de pruebas y ejecuta todos los métodos anotados con `@Test`. El resultado se presenta en formato estandarizado indicando el número total de tests, fallos, errores y tests omitidos. Un resultado ideal muestra `Failures: 0, Errors: 0`, lo que certifica que todas las aserciones se cumplieron.

En entornos de desarrollo integrado (IDE) como VS Code o IntelliJ IDEA, los tests se pueden ejecutar individualmente mediante iconos de reproducción junto a cada método `@Test`, facilitando la depuración incremental. Cuando un test falla, el framework muestra:

1. El nombre del test fallido
2. La aserción que no se cumplió
3. El valor esperado vs. el valor real
4. El mensaje personalizado proporcionado (si existe)

Este diagnóstico preciso acelera la localización de errores y refuerza la filosofía de "fallo rápido": cuanto antes se detecta un defecto, menor es el costo de corrección. Además, la ejecución automatizada permite integrar los tests en pipelines de integración continua (CI), garantizando que cada cambio en el repositorio mantenga la integridad del sistema.

Buenas prácticas para tests mantenibles y efectivos

Cada test debe verificar una única condición o escenario, evitando agrupar múltiples aserciones no relacionadas en un mismo método. Esto asegura que, ante un fallo, se identifique inmediatamente qué comportamiento específico dejó de funcionar. Los tests deben ser independientes entre sí: no deben compartir estado ni depender del orden de ejecución, ya que JUnit no garantiza dicho orden.

Es crucial probar no solo casos típicos, sino también:

- **Casos límite:** valores mínimos/máximos, cero, valores negativos
- **Casos inválidos:** entradas nulas, formatos incorrectos, divisiones por cero
- **Casos de borde:** transiciones entre rangos válidos/inválidos

Por ejemplo, para un método que calcula descuentos:

```
@Test void testDescuentoNormal() { /* 10% sobre 100€ → 90€ */ }
@Test void testPrecioCero() { /* 10% sobre 0€ → 0€ */ }
@Test void testPrecioNegativo() { /* Debe lanzar excepción */ }
@Test void testDescuento100PorCiento() { /* 100% sobre 50€ → 0€ */ }
```

Esta cobertura exhaustiva refuerza las invariantes de clase y documenta explícitamente el comportamiento esperado ante situaciones extremas, constituyendo una especificación viva del sistema.

Ejemplo completo: Tests para una calculadora

```
package ejercicio4;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
```

```
class CalculadoraTest {

    @Test
    void testSumarDosPositivos() {
        Calculadora calc = new Calculadora();
        assertEquals(8, calc.sumar(5, 3), "5 + 3 debe ser 8");
    }

    @Test
    void testSumarPositivoYnegativo() {
        Calculadora calc = new Calculadora();
        assertEquals(2, calc.sumar(5, -3), "5 + (-3) debe ser 2");
    }

    @Test
    void testMultiplicarPorCero() {
        Calculadora calc = new Calculadora();
        assertEquals(0, calc.multiplicar(42, 0), "Cualquier número × 0 debe ser 0");
    }

    @Test
    void testDivisionEntera() {
        Calculadora calc = new Calculadora();
        assertEquals(3, calc.dividir(10, 3), "10 / 3 en entero debe ser 3");
    }

    @Test
    void testDivisionPorCeroLanzaExcepcion() {
        Calculadora calc = new Calculadora();
        assertThrows(ArithmeticException.class, () -> calc.dividir(10, 0),
            "Dividir por cero debe lanzar ArithmeticException");
    }
}
```

Este conjunto de tests demuestra cómo validar tanto comportamientos normales como excepcionales, cubriendo escenarios que refuerzan las invariantes matemáticas de la calculadora. La presencia de tests para divisiones por cero documenta explícitamente que este caso está gestionado mediante excepciones, formando parte esencial de la interfaz pública del método `dividir()`.

Conclusión: Tests como especificación ejecutable

Los tests unitarios trascienden su función verificadora para convertirse en documentación viva y ejecutable del sistema. Al escribir tests antes o simultáneamente al código de producción (TDD), se diseña primero la interfaz pública desde la perspectiva del consumidor, lo que conduce a APIs más cohesivas y encapsuladas. Cada test aprobado certifica no solo que el código funciona hoy, sino que seguirá funcionando tras futuras modificaciones, constituyendo una red de seguridad indispensable en el desarrollo profesional de software.

La inversión inicial en escribir tests se amortiza exponencialmente a lo largo del ciclo de vida del proyecto, reduciendo drásticamente el tiempo dedicado a la depuración manual y permitiendo evolucionar el sistema

con agilidad y confianza. En esencia, los tests bien diseñados son la manifestación práctica del principio de encapsulación: definen qué debe hacer una clase sin revelar cómo lo hace, protegiendo así la integridad del diseño ante cambios inevitables.